

抽象 · 取舍 · 定位 · 开放问题

在自主性、 可追溯性与隔离性 之间做取舍

一个多智能体编排平台的设计论证。本册不讲怎么实现，只回答：这个问题域的本质困难是什么、我用了哪些抽象、这些抽象在已有工作谱系中处于什么位置、哪些问题我没有解决。

本质困难

4 个

核心抽象

8 条

对比对象

18 个

开放问题

5 个

SCOPE · 这一册讲什么

不是「我做了什么」， 是「为什么只能这么做」

一个系统的实现细节说明作者会写代码；一个系统的**约束结构**说明作者理解问题。学术面试考的是后者。本册的每一节都以「这里存在一个无法同时满足的要求」开场，以「我选了哪一边、代价是什么、什么条件下这个选择会失效」结束。

36 页

问题域的本质困难 → 核心抽象与不变式 → 可迁移的设计原则 → 在已有工作中的定位 → 评估方法与五个开放问题。

面试时以这一册为主线。

另一册 · 实现细节参考

53 页

模块、数据表、接口、协议信封、队列策略、具体文件路径。

只在被追问「那你具体怎么做的」时翻。**不要主动讲**——在学术场合先讲实现，等于宣布自己没有抽象层。

■ 三条阅读约定

- **凡是抽象，都给反例。**只说「我做了 X」没有信息量，必须说「不做 X 会怎样」。
- **凡是数字，都给来源。**没有实测过的性能数字一律不写；相关工作的数字标注论文与测量条件。
- **凡是没解决的，写进第五部分。**把局限说成未来工作是包装，说成研究问题才是能力。

一句话主张

LLM Agent 系统的工程难点不在「让模型能调工具」，而在**自主性、可追溯性、可隔离性三者互相拉扯，且无法由单一计算模型同时满足**。本系统的全部结构都是这个约束的产物。

诚实前置

本系统是**工程系统，不是研究原型**：没有对照实验、没有 benchmark、没有量化评估。第五部分逐条列出缺什么、以及如果要做该怎么设计实验。

CONTENTS · 目录

五个部分

I	问题域		IV	在已有工作中的位置	
I.1	四个本质困难	06	IV.1	编排框架谱系	26
I.2	三组无法同时满足的要求	07	IV.2	持久化工作流引擎的对照	27
I.3	设计空间坐标系	08	IV.3	隔离机制谱系	28
I.4	本系统的立场	09	IV.4	协议层：MCP / ACP / A2A	29
II	核心抽象		IV.5	记忆与检索	30
II.1	二元计算模型	11	IV.6	三条定位差异	31
II.2	执行状态与调度的可重建性	12	V	评估与开放问题	
II.3	可追溯性与重放：两条轴	13	V.1	这类系统该怎么评估	33
II.4	循环的有界展开语义	14	V.2	我没有做的评估	34
II.5	类型兼容性作为协商而非拒绝	15	V.3	开放问题 RQ1-RQ3	35
II.6	能力单调收窄不变式	16	V.4	开放问题 RQ4-RQ5	36
II.7	信任域与单向断言	17	V.5	学术向问答	37
II.8	记忆表示的选择判据	18	V.6	三分钟陈述脚本	39
III	可迁移的设计原则				
III.1	归因需求决定配置策略	20			
III.2	fail-closed 的适用边界	21			
III.3	人在回路的粒度选择	22			
III.4	隔离机制的选择函数	23			
III.5	运行时状态必须外置	24			

NAVIGATION

如果只有十分钟：读第 4 页（主张）、II.5-II.7（三条不变式）、V.3-V.4（开放问题）。这三块决定面试官对「有没有科研潜力」的判断。

整套系统压缩成一页

CLAIM

LLM Agent 系统的工程难点，不是让模型能调用工具，而是**自主性、可追溯性、可隔离性三个要求互相拉扯**；它们无法由单一计算模型同时满足，因此必须**分层**：用确定性图承载可追溯与可恢复，用无界循环承载自主推理，用硬件级边界承载隔离，再用**一组跨层不变式**把三者缝合。

LAYER 1

确定性图

不可变发布快照 + 持久化步骤状态。
调度是从持久状态重建的纯函数，
因此进程崩溃不丢调度进度。承载：
可审计、可续跑、可视化。

LAYER 2

无界循环

Agent 回合本身不可预测，只能用**预算**（轮次上限、超时）和**权限门**（工具级审批）截断。对上层暴露为可挂起、可快照、可取消的黑箱。

LAYER 3

硬件边界

模型生成的代码在独立内核的 microVM 内执行，第三方插件在 WASM 内执行。业务进程不持有任何宿主级权限。

■ 缝合三层的三条不变式

INV-1 · § II.6

能力单调收窄

调用链上任意子体的能力集合必须是父体的子集，任何位置都不得放宽。
把「权限正确」从「每处都写对」降为「检查每条边」。

INV-2 · § II.7

不可信方不自证

跨信任域的每个断言都由更高信任域验证：guest 的网络身份由宿主验证，回调身份由管理器验证，插件完整性由规范化内容摘要验证。

INV-3 · § II.2

状态外置于运行时

调度进度、会话快照、审批状态一律落在运行时之外的持久存储。运行时实例是可丢弃的。

可以被追问的最短版本：三层分离让每一层只解决一类问题，三条不变式保证跨层组合不会引入新的失效模式。系统的复杂度主要花在**维持这三条不变式**上，而不是功能数量上。

PART I



问题域

先说清楚这个问题为什么难。如果 Agent 编排只是「把 LLM 调用串起来」，那它不值得做成一个系统，更不值得作为研究对象。它难在四个地方，并且这四个困难两两之间还会互相加强。

I . 1

四个本质困难

I . 2

三组张力

I . 3

设计空间

I . 4

本系统立场

I.1 · INTRINSIC DIFFICULTIES

为什么它不是一个普通的分布式系统

传统工作流引擎（Airflow、Temporal）已经把「分布式任务编排」这件事解决得很好。Agent 系统之所以不能直接套用，是因为它破坏了那些引擎赖以成立的四个前提。

D1 执行是非确定性的，但恢复语义要求确定性

Temporal 这类引擎的可靠性建立在**确定性重放**之上：把历史事件重新喂给同一段代码，必须走出同一条控制流。LLM 调用天然违反这一点——同样的提示词，两次执行可能调用不同工具、循环不同轮数。于是「重放」这个最基本的恢复原语失效了，必须重新定义什么叫「恢复到同一个状态」。

D2 控制流由内容决定，图的形状运行时才知道

静态 DAG 假设「边在提交时已知」。而 Agent 的下一步取决于上一步生成的文本。要么放弃静态结构（退化为自由循环，失去可视化、可审计、可静态分析），要么强行静态化（失去自主性）。这是本领域最核心的表达力冲突。

D3 能力与风险同源，且主体意图不稳定

Agent 的价值随能力单调上升：能读文件 < 能写文件 < 能执行命令 < 能修改自身配置。风险同步上升。经典 RBAC 隐含假设「主体的意图是稳定的、可归因的」，而模型的意图既不稳定也不可归因——它可能被提示词注入改变目标。**权限模型因此不能只绑定主体，必须绑定「这一次具体操作」。**

D4 状态是多源的，「当前状态」没有单一定义

一次 Agent 执行的状态同时分布在：模型上下文窗口、工具产生的外部副作用（文件、HTTP 请求）、沙箱文件系统、待人工审批的挂起点。任何一处遗漏，「断点续跑」就只是幻觉。定义清楚**哪些状态必须持久化、哪些可以重建、哪些根本不可恢复**，是这个系统最基础的工作。

COMPOUNDING

四个困难会互相加强：D2 让图不可预测 → D4 的状态边界随之不可预测 → D1 的恢复语义更难定义 → 为了安全只能在更多地方插入 D3 的权限门 → 人工中断频率上升，自主性收益被抵消。**这个正反馈回路是整个设计的主要敌人。**

I.2 · TENSIONS

三组无法同时满足的要求

■ T1 · 自主性 ↔ 可控性

全自主

模型自行决定一切。失败模式：无限循环烧钱、不可解释的副作用、出事后无法归因。



全人工确认

每步都要点同意。失败模式：中断代价超过自动化收益，用户直接关掉这个功能。

落点：按操作的不可逆性分档，而不是按重要性。只读工具自动放行；可逆写操作记录审计；不可逆或改变系统自身配置的操作需要人工确认，且确认粒度是单次工具调用而非整个回合（§ III.3）。

■ T2 · 确定性编排 ↔ 涌现推理

纯 DAG

可视化、可审计、可续跑。失败模式：表达不了「视情况决定下一步」，退化成穿着 workflow 外衣的脚本。



纯 Agent 循环

能处理开放任务。失败模式：无结构可言，无法被非技术用户理解、无法逐步重放、无法局部重试。

落点：不做统一模型，做二元模型——外层确定性图，内层无界循环，两者通过「可挂起、可快照、可取消的会话」接口耦合（§ II.1）。代价是概念数量翻倍，用户要理解两套心智模型。

■ T3 · 能力开放 ↔ 隔离强度

进程内执行

零开销、低延迟、易调试。失败模式：模型生成的代码与业务服务同权限，一次提示词注入即宿主沦陷。



强隔离

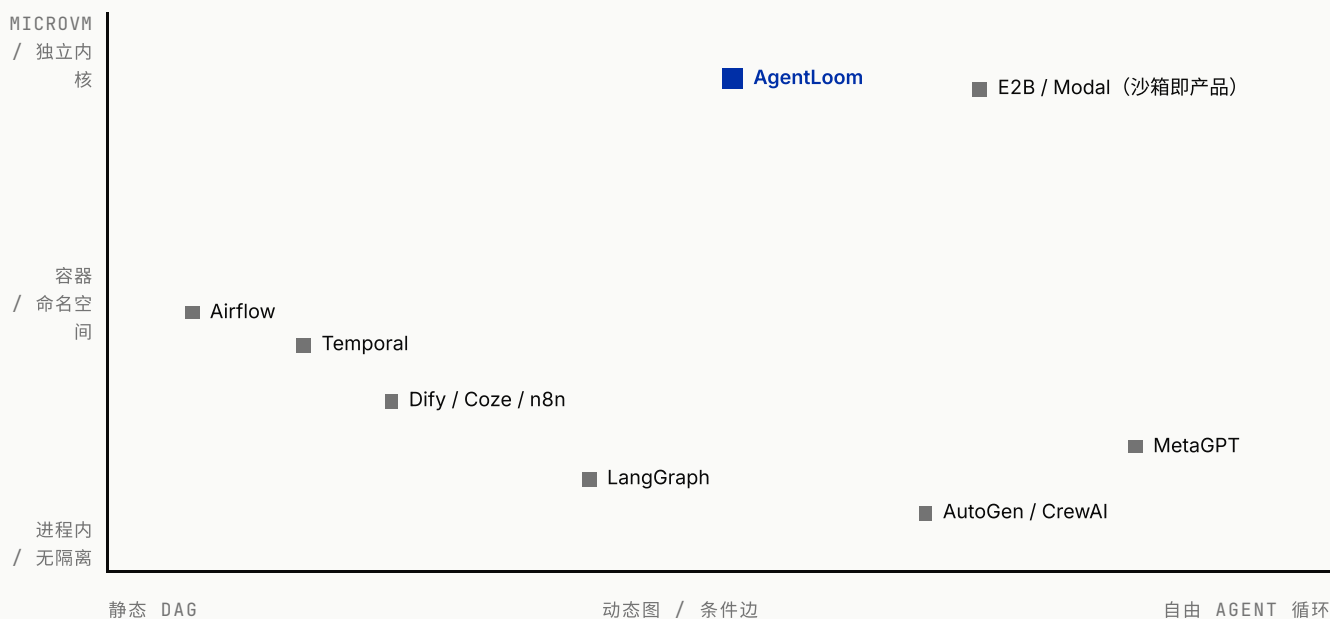
独立内核、独立网络栈。失败模式：启动开销、运维复杂度、需要特权组件，本身成为新的攻击面与单点。

落点：不选一档，而是按代码来源分档：平台自有逻辑进程内跑；第三方插件进 WASM（无系统调用面）；模型生成的代码进 microVM（独立内核）。特权收敛到单一组件（§ III.4）。

I.3 · DESIGN SPACE

两个轴就能把同类系统摆清楚

横轴是**控制流的确定程度**（编排层允许多大的运行时不确定性），纵轴是**执行隔离的强度**（对不可信代码的防护层级）。绝大多数同类系统只在其中一个轴上做深，很少有系统同时处理两轴。



READING

左下角是成熟的确定性工作流引擎——可靠但表达不了自主推理；
右下角是 Agent 框架——表达力强但执行隔离基本靠调用方自觉；
右上角是把沙箱本身做成产品的基础设施，不负责编排。

POSITIONING

本系统落在**中上区域**：编排层保持结构化（可审计、可续跑），但允许节点内部是无界循环；同时自己承担隔离责任而不外包。**这个位置的代价是系统复杂度显著高于单轴系统**，收益是能同时给出「可审计」与「能自主写代码」两个属性。

I.4 · POSITION

四条可以被证伪的立场

每条都写成可反驳的形式——如果反驳成立，对应的设计就应该推翻。

P1 Agent 与 Workflow 应当是并列实体，而不是一方包含另一方

理由：二者的生命周期、恢复边界、复用粒度都不同。工作流执行是一次性的、有确定终点、要求可审计重放；Agent 是长期存在、跨会话继续、需要独立版本与权限策略的实体。

如何证伪：如果能给出一个统一的计算模型，同时提供「逐步可重放」和「无界自主循环」，且概念复杂度不高于两套模型之和，那么这条立场就站不住。

P2 「可追溯」与「可重放」是两件事，必须分开承诺

理由：「记录了发生过什么、且能从中断处接着跑」与「再跑一次结果一样」是两个独立性质。前者不受非确定性影响，后者被非确定性直接摧毁。合在一起讲会导致过度宣称——因为只要条件分支消费了模型输出，连“走了哪些节点”都不保证一致 (§ II.3)。

如何证伪：如果用户实际只关心「最终输出是否一致」，那么为可追溯性付出的持久化成本就是无用的工程负担。

P3 隔离级别应当由代码来源决定，而不是由性能预算决定

理由：威胁模型不同：平台自有代码经过审查，第三方插件经过签名但逻辑不可信，模型生成的代码完全不可信且每次都不同。用同一档隔离服务三类代码，要么过度付费，要么留下缺口。

如何证伪：如果 WASM 的隔离强度对模型生成代码也够用（或 microVM 的开销可忽略），分档就没有必要。

P4 安全性应当表述为少数几条可检查的不变式，而不是一份规则清单

理由：规则清单的正确性等于「每处都记得遵守」，随功能增长必然衰减；不变式可以逐边检查，甚至可能静态验证 (§ V.3 / RQ5)。

如何证伪：如果实际系统中存在大量无法归入这几条不变式的安全逻辑，说明抽象没有覆盖真实约束，只是事后叙事。这一条目前只做到了「大部分安全逻辑可归入三条不变式」，没有做形式化验证。

PART II



核心抽象

八条抽象。判断标准是：去掉它，系统会在哪里塌。凡是去掉之后只是「代码难看一点」的，都不算抽象，只算约定。

II.1-II.2

计算模型与状态

II.3-II.4

追溯 / 重放 · 循环

II.5

类型协商

II.6-II.8

不变式与记忆

II.1 · DUAL COMPUTATION MODEL

外层有界，内层无界

这是全系统的第一性结构：**不试图用一个模型同时表达确定性编排与自主推理，而是承认它们是两类计算，并只定义一个耦合接口。**

OUTER · 有界

确定性图

节点集有限、边在发布时冻结、拓扑序可计算、每个节点有明确的开始与结束。可以回答「跑到哪了」「还剩几步」「重跑哪一段」。

终止性：由图的有限性保证（循环另有预算上限，见 II.4）。

INNER · 无界

轮次不可预知、工具调用序列由内容决定、可能永不自然停止。**不可能从结构上保证终止**，只能从外部施加预算与权限门。

终止性：由预算（最大轮次、超时、令牌上限）与人工中断强制。

耦合接口只有四个操作

```
// 外层看到的内层，是一个黑箱状态机
create(config) → Session // 由不可变快照编译而来
load(snapshot) → Session // 崩溃后从持久状态重建
prompt(session, input) → Stream<Event>
cancel(session) → void
```

这四个操作定义了内外层的**唯一契约**。它带来两个直接后果：

- **运行时可替换**。同一接口下有两个实现——进程内运行时与 microVM 内运行时。编排层完全不知道自己调的是哪一个，因此隔离级别成了部署选择而不是代码分叉。
- **Agent 可以脱离编排单独存在**。因为接口不假设调用者是图调度器，同一个 Agent 也能被对话入口、子 Agent 递归调用复用。

COST OF THIS CHOICE

二元模型的代价是**概念不统一**：用户必须理解两套心智模型，文档要解释两遍，两层的错误处理和事件流也要各写一套。如果面试官问「为什么不像 LangGraph 那样把 Agent 也画成带环的状态图」——那是把两层压成一层的做法，好处是概念统一，代价是外层丧失「有限步内必然终止」这个性质，从而无法给出进度、剩余步数与确定的重跑边界。**这是表达力与可分析性的经典交换。**

II.2 · STATE & SCHEDULING

调度必须是持久状态的纯函数

把一次执行的状态定义为三元组：

$$E = (G, \sigma, \kappa)$$

G = 发布快照，不可变。执行期间不随编辑变化

σ : $NodeId \rightarrow StepState$ (持久化)

κ : $NodeId \rightarrow Checkpoint$ (持久化)

$ready(E) = \{ n \mid \forall p \in pred(n), \sigma(p) \in \{done, skipped\} \}$

$next(E) \subseteq ready(E)$

三个分量全部在运行时之外，于是得到一条性质：

调度状态可从持久存储完全重建。任何一个工作进程在任意时刻崩溃，新进程读库即可继续，不需要恢复内存中的任何数据结构。

■ 这条性质换来了什么

- **无需内存计数器。**「还差几个前驱完成」不缓存在内存里，每次完成事件都重新查询前驱状态，因此进程重启不影响调度进度。
- **调度逻辑可单测。** `next()` 是纯函数，测试不需要起数据库和队列。

■ 但「可重建」不等于「并发安全」

这是两个必须分开讲的性质，混为一谈会被追穿。可重建只说明**状态在外部**；多个工作进程同时推进同一张图，还需要**原子占位**。

本系统的做法是在状态迁移上做**比较并交换**：先读出步骤当前状态，更新时把它写进 `WHERE` 条件 (`id = ? AND status = <刚读到的值>`)，受影响行数为 0 即判定为非法迁移。**因此同一步骤不会被两个调度者同时推入队列**——入队发生在这次条件更新之后。

仍未解决的两点

① 竞争失败的一方**抛异常**而不是当作良性无操作，这会把一次正常的并发去重表现成错误；② 条件更新**之前**写入的东西（如步骤输入）没有被这道闸门保护。**而且我没有做过并发压力测试**，所以只能说「有 CAS 机制」，不能说「已验证并发安全」。

■ 代价

每个节点完成都要多做几次数据库查询。这是**用吞吐换可重建性**——在 LLM 调用动辄数秒的场景里，几毫秒查询可以忽略；如果节点是微秒级计算，这个设计就是错的。

BOUNDARY

「可重建」只覆盖调度层，不覆盖副作用层。若某节点已发出不可逆的外部请求后崩溃，重跑会再发一次。要做到端到端 `exactly-once`，需要节点级幂等键——本系统只有部分节点具备 (§ V.2)。

TRANSACTIONAL BOUNDARY

与之配套的一条工程约束：**任务入队必须发生在事务提交之后**。否则工作进程可能先于数据可见性读到不存在的记录。这是「消息先于数据可见」的经典陷阱，正确解法是事务性发件箱；本系统只做到提交后回调，仍存在「提交成功但入队前崩溃」的窗口。

II.3 · AUDITABILITY ≠ REPLAYABILITY

「可复现」是两件事，混在一起讲必被追问

最容易出错的表述是「我的系统结构上可复现」。**它不成立**：条件节点可以消费上游 LLM 的输出，因此同一份快照、同一份输入，两次执行走的分支和实际参与的节点集合都可能不同。必须把它拆成两条**互相独立**的轴。

本系统达到 A2

A0 只有最终输出

A1 每步输入/输出/工具调用被完整记录，可事后回看

A2 在 A1 之上，执行可从持久状态恢复，并支持从任意节点及其下游局部重跑

它回答的是「**刚才发生了什么**」与「**能不能接着跑**」。与非确定性无关，因此 LLM 的随机性不构成障碍。

边界：A2 成立在编排层（步骤状态与检查点都落库，可重建调度、可从任意节点及下游重跑）。**一个正在进行的 Agent 回合不在此列**——它活在发起进程的内存里，进程崩溃即丢失，只能从该步骤整体重跑。

轴 R · 重放保真度

本系统停在 R0

R0 无重放保证

R1 结构重放：同快照同输入 ⇒ 同节点集合与同分支

R2 完全重放：连输出文本一致

它回答的是「**再来一次还会不会一样**」。**R1 并不比 R2 容易多少**——只要分支依赖模型输出，就必须冻结或回放模型与工具的输出，而那基本等价于实现 R2 的机制。

准确的一句话：本系统提供 **A2 级可追溯与可恢复**，不提供任何级别的重放保证。企业侧真正需要的事事故归因、局部重试、合规审计三件事，全部只依赖 A 轴；R 轴主要服务于研究与回归测试场景。

■ A2 的实现代价

- 图必须冻结成不可变快照 → 需要草稿 / 发布分离与版本表
- 每步的输入必须持久化，而不只是输出 → 存储成本换「它到底收到了什么」
- 事件流必须带单调序号 → 断线后可增量续传，时间线不缺口
- 调度状态必须可从持久存储重建 (§ II.2)

如果被追问「那你怎么做回归测试」

没有 R1 就无法用「重跑一遍比对」的方式做回归。可行的替代是**对模型与工具调用做记录-回放**：把每次调用的请求指纹与响应存进账本，重放时命中账本即返回记录值。这条路径技术上清晰、工作量可估，但会引出「请求指纹如何定义才既不过拟合也不误命中」的问题——这正是 RQ1。

为什么这个区分重要

把 A 轴的成果说成「可复现」是学术场合最典型的过度宣称。**分开讲反而更强**：它说明我知道自己的机制到底保证了什么、边界在哪。

II.4 · BOUNDED UNROLLING

DAG 里怎么放循环

DAG 的定义是无环，而迭代天然有环。三种处理方式，代表三种取舍：

方案	机制	代价
加回边 LangGraph 类	图本身允许环，用状态机语义推进	拓扑序失效；「跑到第几步」无法定义；无法给出剩余进度；静态分析困难
事件溯源 Temporal 类	保留全部历史事件，重放推导当前状态	要求确定性重放，与 LLM 冲突；历史无限增长
有界展开 本系统	循环体是带父子层级的子图，拓扑排序时排除；每轮开始前重置子步骤状态	历史轮次的中间产物不可寻址 ——一个节点只有一行记录，只保留最后一轮的具体状态

为什么选有界展开

- 顶层图保持无环，**拓扑序、进度、剩余步数三个属性全部保住**。
- 每个节点在状态表里恒定一行，存储与查询复杂度不随轮次增长。
- 轮次信息聚合在父节点上，天然形成「循环作为一个可折叠的整体」的视图。

终止性从哪来

不来自结构，而来自**显式预算**：条件循环有最大轮次硬上限（默认 100）。这不是保守工程，而是必需——终止条件由模型判断时，「条件永远为真」是一个真实且昂贵的失效模式，不能只靠提示词约束。

TRADE-OFF, STATED

有界展开牺牲的是**历史轮次的可寻址性**。用户想问「第 3 轮的第 2 个节点输出了什么」，只能去事件流里翻，不能直接寻址。

这是一个可以改进的点：代价是把状态表从「每节点一行」变成「每节点每轮一行」，存储与索引成本随轮数线性增长。**在什么条件下这笔交换划算，取决于实际循环轮数分布——而我没有这个数据。**

II.5 · TYPES AS NEGOTIATION

把类型系统 从拒绝器改成协商器

可视化编排里，「这两个端口能不能连」是一个类型判定问题。常规做法是二值：类型相同则允许，否则拒绝。实际使用中这条规则几乎立刻失效——上游产出 `{userId, name}`，下游需要 `{id, name}`，语义完全一致但结构不同。

本系统把判定结果定义成一个**四级的有序集**，而不是布尔值：

EXACT > TRANSFORM > PARTIAL >
INCOMPATIBLE

EXACT 结构一致，直连

TRANSFORM 存在确定性转换函数

PARTIAL 部分匹配 + 缺失字段 + 候选映射

INCOMPAT 拒绝 + 冲突路径

关键设计：**PARTIAL 允许连接**。缺失字段与候选映射作为「待补证据」持久化在边上，交给用户在配置面板里确认。类型系统从「守门人」变成「协商中介」。

学术定位

这本质上是 **gradual typing** 的思路：允许一部分契约在运行时才被满足，换取开发期的可用性。代价与 gradual typing 一样——把一部分错误从编译期推迟到运行期。

两阶段判定

交互层面还有一个约束：拖拽连线的回调是同步的，不能等待异步判定。于是拆成两阶段——同步的廉价规则先挡明显错误保证跟手，异步的结构化 schema 深比较给出权威结论。这与编译器的「快速语法检查 + 完整类型检查」分层同构。

WEAKEST LINK

候选映射依赖**字段名相似度启发式**。它没有 ground truth、没有评估集、没有准确率数字。这是全系统中最像「研究问题」也最缺乏验证的部分（RQ3）。

II.6 · INVARIANT 1

能力单调收窄

对调用链上的任意父子关系 ($p \rightarrow c$):

$$\text{cap}(c) \subseteq \text{cap}(p)$$

且不存在任何运行时路径可以放宽 cap 。

一条约束，四处体现。它的价值不在于「更安全」，而在于把**权限正确性**从一个全局性质降级为一个局部可检查的性质。

■ 四处体现

- **子 Agent 不能自带沙箱**。它并入父的执行环境，而不是新开一个。否则一次对话可能拉起十几个虚拟机，且父子文件系统割裂。
- **低隔离父体不能调用高隔离子体**。进程内运行时没有能力托管一个 microVM 生命周期，这类调用在发布期就被拒绝。
- **插件运行时配置只能收紧**。平台设定超时、内存、可访问主机的上限，插件可以要求更少，不能要求更多。
- **沙箱内的 guest 不能声明自己的权限**。能力集由外部注入，guest 侧的任何自述都不被采信（见 II.7）。

■ 为什么这是抽象而不是规则

如果写成规则清单——「子 Agent 不要开沙箱」「插件别放宽超时」——那么正确性等于「每个新功能的作者都记得这几条」，必然随时间衰减。

写成不变式之后，检查对象变成**调用图上的每一条边**：只要每条边都满足 $\text{cap}(c) \subseteq \text{cap}(p)$ ，全局性质自动成立。这是一个可以在代码审查中机械执行、原则上也可以静态验证的判据。

一句话答面试：我没有让系统「更安全」，我让「是否安全」变成了一个可以逐边判定的问题。

NOT VERIFIED

当前这条不变式靠人工遵守与测试覆盖，**没有静态检查器**。给定系统的能力集合与调用图，自动验证该不变式是否处处成立，是一个明确可做的工程—研究结合点（RQ5）。

II.7 · INVARIANT 2

不可信方永远不自证

对任意跨信任域的断言 a ("我是谁"、"我能做什么"、"这份数据没被改"):

$\text{verifier}(a)$ 的信任级别 $>$ $\text{claimant}(a)$ 的信任级别

四个信任域

客户端 持用户凭证与本地私钥	USER
控制面 业务服务，普通权限	CONTROL
运行时管理器 唯一特权组件	PRIVILEGED
沙箱 guest 完全不可信	UNTRUSTED

三个实例

- 网络身份。** guest 声称自己的 IP/MAC 毫无意义——宿主侧在虚拟网卡入口做流量过滤，同时校验源 IP、MAC 与 ARP 发送方地址，不匹配的帧直接丢弃。这是把「网络地址」变成「不可伪造的身份」。
- 回调身份。** guest 拿不到真实的回调地址与上游令牌，只拿到一个不透明中继地址；管理器按来源地址与不透明令牌双重校验后，代持真实凭证转发到白名单主机。
- 制品完整性。** 插件签名不签 ZIP 字节，而签一份**规范化内容描述**（排序后的清单 + 逐文件哈希）。因为容器格式的字节表示不唯一，签字节等于把签名绑在打包工具上。

GENERALIZATION

这三个实例表面上毫无关系——一个是网络、一个是鉴权、一个是供应链。但它们是同一条不变式的三次应用：把「相信对方说的」替换成「由更高信任级别的一方独立验证」。

能把三件事收敛成一条规则，说明抽象抓对了。反过来，如果某处安全逻辑无法归入这条规则，就应该怀疑那里存在未识别的信任假设。

COROLLARY

推论：**密钥不跨域下发**。控制面不持有签发 guest 身份的私钥，guest 也不持有控制面的客户端证书。任一侧被攻陷都无法伪装成另一侧。

II.8 · MEMORY REPRESENTATION

相似性检索 解决不了关系性问题

系统里同时存在两套「让模型记住东西」的机制，用了两种完全不同的表示。这不是历史包袱，是按**检索目标**分的。

用途	表示	检索	判据
知识库	文本分块 + 向量	向量近邻 + 重排	目标是 相似 ：找和问题语义接近的原文片段。规模万级以上，无需版本管理
Agent 记忆	带父子边的结构化图 + 版本 + 披露级别	全文检索 + 图遍历	目标是 关系 ：「项目 A → 模块 B → 约定 C」。规模百条量级，需要版本、审阅与作废

■ 为什么记忆不用向量库

- **结构本身就是信息**。层级与依赖关系无法由向量相似度表达，而它恰恰是复用经验时最有用的部分。
- **记忆会过时，需要作废而非删除**。版本 + 审阅状态让错误结论可被标记废弃并保留追溯，这在纯向量集合里很难表达。
- **需要披露控制**。不同场景看到不同深度的记忆。
- **规模不匹配**。百条精炼结论用全文检索足够，引入向量库只增加一个外部依赖与一致性问题。

要谨慎表述

不能说「图谱记忆优于向量检索」。现有证据是**条件性**的：GraphRAG 的论文把向量 RAG 定位为「局部事实强、全局归纳弱」，并报告在其两个语料上全局性问题的全面性与多样性占优，但**直接性 (directness) 上向量方法更好**，且图抽取与社区摘要需要额外的 LLM 索引成本。目前没有跨语料、统一模型、统一成本口径的证据表明结构化图谱全面胜过向量。

FRAMING

正确说法是：**两种表示服务两类查询**。本系统按查询类型分配表示，而不是押注某一种表示更好。

PART III



可迁移的 设计原则

这一部分刻意脱离本系统。每条原则都写成「给定什么条件、应该选哪个方案」的判据形式——如果一条经验只在自己的项目里成立，它就不是原则，只是习惯。

III.1-III.2

配置策略 · 失败姿势

III.3-III.4

中断粒度 · 隔离选择

III.5

状态外置

III.1 · CONFIGURATION DRIFT

同一个问题， 两个相反的正确答案

问题：一个长期存在的执行体，在运行过程中它依赖的配置被改了。应该沿用旧配置，还是切到新配置？

CASE A · workflow 执行

钉死快照

执行开始时冻结图快照，中途不受编辑影响。

理由：这条执行要被审计、要能局部重跑。如果图中途变形，「重跑第 3 步」就失去意义——你重跑的是另一张图的第 3 步；事故归因时也说不清「当时到底跑的是哪张图」。

自动跟随最新

继续对话时检测到发布版本变化，就地重建运行时并切到新配置，保留历史消息。

理由：对话的产物不承诺可重放，钉死版本换不来任何重放能力，却会让 bug 修复永远到不了老对话。

判据（可迁移）：配置漂移策略由该执行体是否需要**事后归因与局部重跑**决定，而不是由「一致性听起来更好」决定。

需要归因 / 局部重跑 → 钉死快照，漂移是污染。

只需要「下一步行为正确」→ 跟随最新，钉死是纯粹的成本。

■ 推论：草稿号 ≠ 版本号

一旦决定「执行读快照」，就必然需要两个独立的版本概念：一个是编辑期的**修订号**（兼作乐观并发控制的版本，每次自动保存递增，会涨到几百），一个是发布期的**发布号**（只在快照首次发布时分配，用户可见）。

把两者合一是一个常见错误：要么自动保存污染版本号，要么并发编辑检测失去依据。

GENERALIZATION

这个判据不限于 Agent 系统。CI 流水线定义、特征工程管线、报表调度——凡是「长时间运行 + 定义可编辑 + 需要事后归因」的系统，都会遇到同一个问题，且答案取决于同一个变量。

FAILURE MODE

最差的做法是**不做决定**：执行时随手读当前定义。表现为「同一次执行的前半段和后半段用了不同的图」，而且这种 bug 只在编辑与执行并发时出现，极难复现。

III.2 · WHEN TO FAIL CLOSED

失败时应该 停住还是继续

「fail-closed（出错就拒绝）比 fail-open 安全」是一句正确但过于粗糙的话。真实判据是**两类错误的代价谁更大**。

设 e_1 = 误拒代价（本该放行却停住）， e_2 = 误放代价（本该停住却放行）

$e_2 \gg e_1 \rightarrow$

fail closed

$e_1 \gg e_2 \rightarrow$

fail open + 告警

$e_1 \approx e_2 \rightarrow$

降级到受限模式，不做二选一

■ 本系统里的三档

场景	选择	理由
沙箱冷恢复时清理失败	fail closed	误放的后果是两个虚拟机抢同一块可写磁盘 → 数据损坏。宁可留一个需人工处理的资源
发布期检测到不合法的运行态 / 工具组合	fail closed	能在发布期发现的问题留到执行期，成本高一个数量级
权限确认超时	fail closed	超时默认放行等于取消了这道门
向量检索后端不可用	降级	回落到词法检索：质量下降但功能可用，误拒代价（整个知识库不可用）远大于误放
类型判定引擎加载失败	降级	回落到同步规则判定：宁可判得粗，也不能让用户连不了线

反例：一处我选错了

静态数据加密在**加密失败时会降级写明文**。这里 e_2 （机密数据以明文落库，而用户以为它是加密的）显然大于 e_1 （这次执行失败，用户重试）。按上面的判据应当 fail closed，或至少交由组织策略决定。**这是一个已识别但未修的设计缺陷**，主动讲出来比被问出来好。

SECOND-ORDER

fail-closed 有个二阶代价常被忽略：它把**可用性风险转移到了检测逻辑本身**。如果检测器有误报，fail-closed 会把误报直接变成故障。所以严格的 fail-closed 必须配一条人工旁路，否则一次误报就是一次事故。

III.3 · HUMAN-IN-THE-LOOP GRANULARITY

在哪一层拦住模型

人在回路的设计，真正的自由度不是「要不要审批」，而是**审批的粒度**。粒度决定了两件事：中断频率，以及恢复时需要重建多少状态。

粒度	暂停的对象	恢复单元与成本	问题
run 级	整次执行	低（重新开始）	审批点太粗，用户看不到「到底在批准什么」
turn 级	整个模型回合，步骤状态置为等待	高：要重建整个 agent loop 的状态机	暂停语义与失败语义混淆，监控上分不清「挂了」还是「在等人」
tool 级 本系统	单次工具调用； 步骤仍为运行中	低：模型回合本就在进行，批准后原地继续	需要跨进程共享挂起状态（发起方与接收审批的进程通常不是同一个）

■ 为什么 tool 级是对的

因为等待的对象本来就是一次工具调用，不是整个回合。把状态标在回合上是一次不必要的抽象泄漏：模型的这一轮仍在进行中，只是其中一个动作被冻结了。

由此得到一个反直觉但正确的结论：**「等待审批」不应该是步骤状态，而应该是工具调用状态；步骤本身仍是「运行中」**。监控面板据此能正确区分「卡住了」和「在等人」。

真正该问的维度

不是「有没有工具级审批」，而是**审批点的粒度与恢复单元的粒度**是否一致。二者不一致时（审批在工具、恢复靠重跑整个节点），暂停前的副作用会被重复执行——**审批本身反而引入了新的正确性风险**。

■ 中断预算：一个应该被度量却没人度量的量

审批设计里真正稀缺的资源是**用户的注意力**。可以定义：

$$\text{中断率} = \text{需人工确认的工具调用数} / \text{总工具调用数}$$

它必须随任务规模**次线性**增长，否则自主性的收益会被中断成本吃光。本系统的处理是按操作不可逆性分档，并允许按「会话 × 操作类别」记住决定。

不要 OVERCLAIM

tool 级审批不是本系统独有：OpenAI Agents SDK 的 `needs_approval` 精确到单次工具调用，运行状态可序列化后在另一进程恢复；LlamaIndex Workflows 也在工具内发事件等待人类响应。真正的差异在于**把审批状态与步骤状态解耦**（步骤仍为运行中，批准后不重跑任何已执行代码），以及**审批决策可按「会话 × 类别」固化并写审计**。

要同时说出局限：**等待发生在发起进程的内存里**，Redis 只负责把「决定」从接收审批的进程传回来，**发起进程一死这次回合就丢了**——这一点上带外部检查点的 LangGraph 反而更强。

NO DATA

我没有测过实际中断率，也没有量化过「记住决定」对它的影响。这是一个容易设计、代价很低的实验 (§ V.2)。

III.4 · CHOOSING AN ISOLATION MECHANISM

按代码来源分档， 不按性能预算分档

选择函数的输入不是"要多快"，而是三个问题：

1. 这段代码的**来源**是否可审查？
2. 它需要的**接口面**有多宽（纯计算 / 文件 / 网络 / 完整系统调用）？
3. 它**每次都不同**吗？（一次性生成 vs 长期复用）

代码来源	隔离层级	接口面	理由
平台自有逻辑	进程内	全部	经过代码审查与版本控制，威胁模型等同于普通服务端代码
第三方插件 已签名，逻辑不可信	WASM	无默认系统调用 ；无文件系统；网络需显式声明主机白名单；固定内存与超时上限	它需要的能力本来就窄（数据转换），窄接口面正好匹配；启动开销远低于虚拟机，适合高频调用
模型生成的代码 完全不可信，每次不同	microVM 独立内核	完整开发环境：文件系统、终端、进程、受控出网	需要完整系统调用面，WASM 无法满足；而共享内核意味着攻击面是整个系统调用接口

严谨表述

不要说「容器不安全」。准确说法是：**容器的隔离边界建立在共享宿主内核之上，攻击面是整个系统调用接口；microVM 把攻击面收敛到虚拟设备接口与虚拟机监控器**。这是攻击面宽度的差异，不是「安全 / 不安全」的二分。

没有实测数字

启动延迟与内存开销的量级请引用原始文献的测量条件，**不要用本系统的经验数字**——我没有做过基准测试。

■ 被忽略的第四档：特权收敛

选完隔离机制还有一个独立决策：**谁来创建这些沙箱**。如果业务服务自己持有创建虚拟机的能力（设备节点、网络管理权限），那么业务层的任意一个远程代码执行漏洞都会直接升级为宿主沦陷——隔离机制再强也没用。

正确做法是把这些特权收敛到一个**只说受限协议、只接受白名单调用的独立组件**。代价是这个组件成为单点：它需要独占本地状态，因而难以水平扩展。这是「攻击面」与「可用性」之间一次明确的交换。

III.5 · EXTERNALIZED STATE

运行时实例必须是可丢弃的

Agent 运行时库（无论自研还是第三方）本质上是一个**内存中的活对象**：它持有对话历史、工具集、进行中的流。进程一挂，全没了。

所以系统必须回答：**谁负责持久化**？三种答案，只有一种可扩展。

A 让运行时库自己持久化

要求库知道你的数据库、你的租户模型、你的事务边界。这会让一个通用库耦合到具体业务，且几乎不可能同时适配多种存储。

B 不持久化，崩溃就重来

对于单次几秒的调用可以接受；对于跑了二十分钟、已经写了文件、已经花了钱的 Agent 回合，不可接受。

C 外面套一层持久化适配器 ← 本系统

运行时库只管活对象；外层适配器负责会话快照、检查点与事件账本，并在需要时用快照重建一个新的活对象。

THE SEAM

这个设计的代价必须说清楚：**它引入了两份状态，且必须保证它们一致**。活对象里的状态和持久化快照之间的同步点选在哪、哪些事件必须落盘、重建之后语义是否等价——这是本系统最容易出 bug 的接缝。

它也解释了为什么「恢复」不是免费的：恢复的不是进程，而是**语义**。

判据（可迁移）

凡是「长时间运行 + 有外部副作用 + 由第三方库承载核心循环」的系统，都应采用 C：**把库当成可丢弃的计算单元，把状态放在自己能控制的地方**。这与「无状态服务 + 外部会话存储」是同一条原则，只是这里的「会话」复杂得多。

PART IV

IV

在已有工作 中的位置

这一部分的目的不是证明本系统更好——它在多数维度上不更好。目的是**准确说出差异在哪一维**。学术面试最忌讳的两件事是：把别人已有的能力说成自己的创新，以及说不清自己和别人到底哪里不同。

IV.1-IV.2

编排框架·持久化引擎

IV.3-IV.4

隔离机制·协议层

IV.5-IV.6

记忆检索·定位差异

IV.1 · ORCHESTRATION FRAMEWORKS

四个维度上的实际差异

以下均据官方文档核实，非印象。中断粒度一栏尤其重要——我原以为工具级审批是差异点，核实后**不是**。

系统	计算模型	状态与恢复	人在回路的粒度
LangGraph	带条件边的状态图，显式支持循环；Pregel 式 super-step 推进	checkpointer 在 super-step 边界 存图状态，thread_id 作为持久游标；内存版进程重启即丢，Postgres/SQLite 版可跨进程	interrupt() 可以写在工具函数里，因此 审批点能到工具调用 ；但官方明确检查点只存在 super-step 边界，恢复时 包含它的整个节点从函数开头重跑 ，暂停前的代码与副作用会再执行一次（节点内 task 结果可被缓存跳过）
OpenAI Agents SDK	Agent + 工具循环，Runner 驱动	RunState 可序列化为 JSON，在 另一个进程 反序列化后继续；Session 存储支持 SQLite / Redis / SQLAlchemy 等	单次工具调用级 ：needs_approval=True，暂停后 RunResult.interruptions 给出待批项，approve / reject 后恢复
LlamaIndex Workflows	事件驱动：step 消费/产出事件；分支即返回不同事件，循环即返回被前序 step 消费的事件	默认 易失 ；Context.to_dict/from_dict 手动做检查点，可跨进程。恢复时重派未完成事件、已完成 step 不重跑；中途捕获时该 step 从头重跑（at-least-once）	工具调用级 ：工具发出 InputRequiredEvent，等待 HumanResponseEvent
CrewAI	Flows 事件驱动，支持状态、循环、分支	@persist 默认落 SQLite，按状态 UUID 恢复，可 fork	任务 / 回合级 ：Task(human_input=True) 在 agent 交付最终答案前请求输入， 不是每次工具调用 ；较新版本的 Flow 方法装饰器可做方法级审批并按 approved / rejected / needs_revision 分支
AutoGen / AG2	事件驱动 actor（Core）+ 对话驱动群聊（AgentChat）	save_state / load_state 导出导入可序列化状态； 不是 内置的持久化工作者/检查点服务	AG2 可在工具内直接请求人类输入并由钩子决定交互通道
MetaGPT	角色化 SOP 协作（PM / 架构师 / 工程师）	官方 README 未承诺检查点 / 断点续跑	未见官方保证

必须修正的自我认知 + 一个更好的切分

工具级审批**已经是成熟做法**（OpenAI Agents SDK、LlamaIndex Workflows 都有），讲成自己的创新会被当场戳破。但核实过程中浮现出一个更精确的维度：「**审批点的粒度**」与「**恢复单元的粒度**」是**两回事**。LangGraph 允许把审批点放进工具，恢复单元却是整个节点（暂停前的副作用会重跑）；OpenAI Agents SDK 两者都在工具调用级。**真正决定用户体验与正确性的是后者**——恢复单元越大，重复副作用的风险越高。这个切分比「有没有工具级审批」有信息量得多。

IV.2 · DURABLE WORKFLOW ENGINES

为什么不直接用成熟的工作流引擎

这是最容易被问、也最能体现是否真的想清楚的问题。Temporal 与 Airflow 在可靠性上远比任何 Agent 框架成熟——不用它们，必须给出技术理由而不是「想自己写」。

TEMPORAL

确定性重放的代价

官方要求：**Workflow 代码必须是确定性的**——给定相同输入，必须以相同顺序发出相同的 Workflow API 调用，否则重放会报非确定性错误。因此所有不确定操作（数据库、外部 API、LLM 调用）都必须放进 Activity，其结果记入事件历史，重放时不重新执行。随机数与本地时间也必须走 SDK 提供的接口。

注意别说错：「Temporal 要求图必须写在代码里」是**错的**。完全可以写一个固定的、确定性的解释器型 Workflow 去执行一份 JSON 图，把每个节点的副作用放进 Activity——这是成熟做法。所以不能用「产品形态冲突」当理由。

AIRFLOW

真环是非法的

源码级事实：DAG.check_cycle() 用三色 DFS 检测回边，发现即抛 AirflowDagCycleException。**真正的环不是合法 DAG。**

替代手段是**动态任务映射**：运行时根据上游输出的列表 / 字典展开 n 个独立任务实例，调度器在执行前创建它们。这是**有限扇出**，不是把下游边指回上游，**无法表达次数未知的循环**。

结论：Airflow 的模型与本系统的「有界展开」（§ II.4）其实同源——都用有限展开替代真环。差别在于本系统把展开发生在**运行时且由模型决定终止**，因此必须额外引入硬预算。

诚实版本（推荐这样答）：「Temporal 能提供比我强得多的持久化保证，而且解释器型 Workflow 完全可以承载可视化 JSON 图，所以技术上没有不可逾越的障碍。我没用它，真实原因有三条：① 调度器在引入它之前已经存在，替换是净成本；② 它引入一整套额外基础设施与运维面，而这是个自托管、私有化部署为主的系统；③ 事件历史与版本化语义会成为长期约束—— workflow 定义一旦变更就要处理 versioning，而我的图是用户天天在改的。这三条都是权衡，不是技术不可行。如果重做，我会认真评估直接构建在其上。」

N8N 的参照价值

n8n 的队列模式是这样分工的：主进程只负责触发，执行 ID 进 Redis，工作进程从数据库取 workflow 定义后执行，结果写回数据库。**这与本系统的「状态外置 + 无状态工作进程」是同一套结构**——说明这个结构不是我发明的，而是这类系统的收敛解。

IV.3 · ISOLATION LINEAGE

四档隔离，按攻击面宽度排列

学术场合谈隔离，正确的语言是**攻击面宽度**，不是「安全 / 不安全」。下表按「不可信代码能直接触达的接口」排序。

机制	边界在哪一层	不可信代码直接触达的接口
进程 / 命名空间 普通容器	共享宿主内核 + 命名空间 与 cgroup 隔离	整个系统调用接口 （Linux 数百个系统调用及其参数空间）。seccomp 可缩小，但缩小到多少取决于配置
用户态内核 gVisor	在用户态实现一层内核，拦截并代理应用的系统调用	应用面对的是用户态内核；真正落到宿主内核的系统调用集合被显著收窄。代价是系统调用密集型负载的性能损耗
硬件虚拟化 Firecracker / Kata	KVM 之上的轻量虚拟机， 每个沙箱独立内核	虚拟设备接口 + 虚拟机监控器 。Firecracker 刻意只实现极少量 virtio 设备，正是为了压窄这一面
语言虚拟机 WASM	线性内存 + 无环境权限的执行模型	只有显式注入的宿主函数 。默认没有文件系统、没有网络、没有系统调用。WASI 会重新引入一部分接口，等于自愿放宽

为什么本系统同时用两档

WASM 的接口面最窄，但它**装不下一个开发环境**——模型生成的代码要读写文件、开终端、装依赖、跑测试。所以：

- 第三方插件 → WASM：它需要的能力本来就窄（数据转换），窄接口面正好匹配，且启动开销低，适合高频调用。
- 模型生成的代码 → microVM：需要完整系统调用面，只能靠独立内核来收窄真正的攻击面。

这不是「更安全」，是把每类代码放到接口面刚好够用的那一档。

数字纪律

Firecracker 的启动延迟与内存开销有公开论文（NSDI 2020, *Firecracker: Lightweight Virtualization for Serverless Applications*）给出的数字，但那些数字有特定的测量条件（镜像、内核配置、硬件）。**我没有在自己的部署上做过基准测试**，因此面试中**不引用任何具体毫秒数或 MiB 数**，只讲攻击面结构差异。被问到性能，正确回答是「我没测过，如果要测我会这样设计实验」。

同类基础设施

E2B、Modal、Daytona 这类「Agent 沙箱即服务」把同一层能力做成了产品。**如果只需要沙箱，直接用它们更理性**；本系统自建的理由是沙箱要与编排、租户治理、审计深度耦合（会话与执行的双向绑定、工作区快照、权限回调），而这些耦合点很难跨越服务边界。

IV.4 · PROTOCOL LAYER

三个协议，三种关系

这一页的主要价值是**避免说错**——「ACP」是两个不同的东西，混淆会立刻暴露没读过一手材料。

MCP (Model Context Protocol, Anthropic) —— Agent ↔ 工具与上下文	TOOLS
ACP (Agent Client Protocol, Zed) —— Agent ↔ 编辑器客户端，定位类比 LSP	CLIENT
A2A (Agent2Agent, 现属 Linux Foundation) —— Agent ↔ Agent	PEER
另一个 ACP (Agent Communication Protocol, IBM / BeeAI) —— 也是 Agent↔Agent，与 Zed 的 ACP 同名但不同物，其仓库已标注并入 A2A	▲ 易混

■ MCP 的安全假设值得单独讲

MCP 规范当前的标准传输是 **stdio** 与 **Streamable HTTP**（后者自 2025-03-26 起取代早期的 HTTP+SSE；SSE 现在只是 POST 响应里的可选流）。

官方安全文档写得很直白：**本地 MCP 服务器可以执行任意代码，且与客户端同权限**；一键配置前必须向用户展示完整启动命令，并建议沙箱化与最小权限。授权规范也明确 OAuth 流程适用于 HTTP 传输，stdio 不走该流程、凭据从环境获取。

本系统的对应决策

进程内运行的 Agent **只允许 HTTP 传输的 MCP**，**stdio 在发布期直接拒绝**。

理由要说准：**不是「stdio 本身不安全」**，而是——在一个多租户服务端里，stdio 意味着**由租户可编辑的配置决定服务进程去 spawn 什么本地命令、注入什么环境变量**。这等于把配置写入权限提升为服务进程权限的远程代码执行。而 MCP 官方文档恰好把「本地服务器与客户端同权限、应当沙箱化」写成了明确建议。

沙箱模式下 stdio 是允许的——因为那时「客户端权限」本身就已经被限制在一个独立内核的虚拟机里。这正是 § 11.6 能力单调收窄的一个实例。

版本纪律

MCP 规范按日期版本演进（本系统实现时对齐的是当时的稳定版）。面试中提到传输或能力细节时**务必带上规范版本日期**，否则容易和对方记忆中的另一个版本对不上。

IV.5 · MEMORY & RETRIEVAL

不要说「图谱比向量好」

这是本领域最容易被抓住的过度宣称。现有证据是**条件性**的，而且几乎每一项都有明确的实验边界。

01 GraphRAG (Microsoft, arXiv:2404.16130)

机制：LLM 抽取实体图 → Leiden 层级社区 → 自底向上生成社区摘要 → 查询时 map-reduce。定位是**全局归纳型问题** (query-focused summarization)，不是局部事实查找。

论文在其两个语料上报告：全面性与多样性优于向量 RAG，**但直接性 (directness) 上向量方法更好**，且图构建与社区摘要需要额外的 LLM 索引成本。**该文长期标注为预印本 / 微软研究报告，不要说成已被某会议录用。**

02 Zep / Graphiti (arXiv:2501.13956)

时序知识图谱式的 Agent 记忆：原始 episode 无损层 → 语义实体与边 → 社区摘要；边带有效区间与失效机制；检索是 **全文 + 向量 + 图遍历的混合**再重排。

要点：它的卖点是「动态增量 + 时间有效性 + 混合检索」，不是「图取代向量」。论文数字为厂商自评，且其中一个基准的部分类别上表现是下降的。

03 MemGPT / Letta (arXiv:2310.08560) 与 Generative Agents (UIST '23)

MemGPT：把上下文窗口当作「主存」，用函数调用在主上下文与外部存储间分页与驱逐，属**虚拟内存式**的记忆管理。

Generative Agents：memory stream + reflection (周期性把经验综合成高阶记忆) + retrieval (时近性、重要性、相关性加权)。

论文本身就报告了记忆漏召回与润色式幻觉。

本系统的立场 (可安全表述)

「我按**查询类型**分配表示，而不是押注某种表示更好：知识库要找相似片段，用向量；Agent 记忆要表达关系、需要版本与作废、规模只有百条量级，用结构化图 + 全文检索。**我没有做过两种表示的对照实验**，所以不声称任何一方更优。」

IV.6 · WHAT IS ACTUALLY DIFFERENT

三条能被追问的差异

核实完一圈之后，能站得住的差异只有这三条。其余部分——工具级审批、检查点、事件驱动、状态外置——都是这个领域的**收敛解**，不是我的创新。

DIFF 1

「原地续行」的粒度：与 LangGraph 各强一轴

工具级审批已经普遍存在，不是差异。真正的差异在**恢复单元**：LangGraph 的审批点可以放进工具，但恢复时**包含它的整个节点从头重跑**，暂停前的副作用需要自己保证幂等。本系统的审批只冻结那一次工具调用，模型回合仍在原地运行，批准后**不重跑任何已执行的代码**。

但必须同时说出反面（我核对过源码）：这只是「**活着的进程内原地续行**」，不是故障恢复。自进化那条门是发起进程内的一个内存 Promise，Redis 只负责把「决定」从接收审批请求的那个进程传回来（发起进程每 250ms 轮询）；沙箱那条门更简单，挂起表是适配器里的一个本地 Map，没有任何分布式同步。**发起进程一崩，这次回合就没了，没有工具级的故障恢复**。而 LangGraph 配合外部检查点存储可以在**另一个进程**继续。

结论要这样讲：两者各强一轴——我在**续行粒度**上更细（批准后不重跑任何已执行代码），它在**跨进程存活性**上更强（进程死了还能接着跑，代价是重跑整个节点）。**正确目标是两者兼得，我只做到了一半**。

DIFF 2

把执行隔离当成一等公民而非附加层

多数编排平台把代码执行隔离作为可选组件，并在文档中标注其局限——例如 n8n 官方明确说任务运行器是用户代码的**唯一**隔离层，而内部模式与主进程同用户运行、可触及数据库与凭据，生产环境应使用外置模式；Dify 的代码节点则运行在阻断文件系统、外网与系统命令的沙箱里。Agent 框架（LangGraph、AutoGen、CrewAI）本身是库，隔离由调用方负责。

本系统把隔离级别做成**按代码来源分档的强制属性**，并把创建沙箱的特权收敛到唯一的独立组件，业务进程不持有任何宿主级权限。**代价明确**：该组件是单点，难以水平扩展。

DIFF 3

把安全性表述为少数几条逐边可判定的不变式

大多数系统的权限模型是规则集合；本系统尝试收敛成三条不变式（能力单调收窄、不可信方不自证、状态外置），使「是否安全」在原则上可以逐边检查，而不是依赖每个功能作者的记忆。

诚实边界：这三条**没有被形式化验证过**，目前只是「大部分安全逻辑可以归入其中」。把它变成可自动验证的性质，正是 RQ5。

一句话定位：本系统不在计算模型上创新——它在计算模型上比同类更保守（坚持顶层无环）。它的位置是：**用表达力换取可分析性，并把执行隔离与多租户治理纳入同一套抽象**。这个组合在现有开源工作里不常见，但它是工程选择，不是研究贡献。

PART V



评估与 开放问题

这是全册最重要的一部分。工程系统在学术场合的价值，不在于它跑起来了，而在于它**暴露了哪些还没有被解决的问题**，以及作者能不能把这些问题表述成可研究的形式。

V.1-V.2

该怎么评估·我没做什么

V.3-V.4

五个开放问题

V.5-V.6

问答·陈述脚本

V.1 · HOW TO EVALUATE

这类系统 该拿什么指标衡量

现有 Agent 评测（工具调用准确率、任务完成率、各类 benchmark）衡量的是**模型能力**。而编排平台的质量与模型能力基本正交——换个更强的模型，平台的隔离强度、恢复能力、中断成本都不会变。这里缺一套面向**系统**的评估口径。

M1 恢复保真度 · Recovery fidelity

怎么测：故障注入。在执行的随机时点杀死工作进程，恢复后比较「继续执行的结果」与「不中断执行的结果」在**可观测契约**上是否等价——完成的节点集合是否为超集、外部副作用是否重复、审批状态是否丢失。

指标：副作用重复率、状态丢失率、恢复成功率。

为什么重要：这是唯一能把「支持断点续跑」从口号变成数字的方法。

M2 隔离强度 · Escape resistance

怎么测：构造对抗性提示词集合，诱导 Agent 尝试逃逸动作——访问宿主元数据地址、扫描同网段其他沙箱、读取沙箱外路径、发起未授权回调、伪造源地址。统计被拦截比例与拦截层级。

指标：逃逸尝试拦截率、最深突破层级。

为什么重要：把「我们做了隔离」变成「在这 N 类攻击下拦截率是多少」。

M3 中断经济性 · Interruption economics

怎么测：在固定任务集上统计中断率（需人工确认的工具调用 / 总工具调用），并观察它随任务复杂度的增长曲线；对比开启与关闭「记住决定」两种配置。

指标：中断率、中断率对任务规模的增长阶（希望是次线性）、误批准率。

M4 协商收益 · Negotiation yield

怎么测：构造端口对数据集（人工标注「应该允许 / 应该拒绝 / 需映射」），测量四级判定的准确率，以及 PARTIAL 候选映射的 top-k 命中率。

指标：判定准确率、映射建议命中率、用户手工修正率。

缺什么

M1-M4 都没有**公开数据集或标准协议**。这本身就是一个可以做的贡献：为「工作流—Agent 混合系统」定义一组系统级评测任务与故障注入协议。目前学界的注意力几乎全在模型能力评测上。

V.2 · WHAT I DID NOT MEASURE

诚实清单

按「说不出数字」的严重程度排序。被问到任何一条，正确回答是承认 + 说明该怎么测 + 说明为什么当时没测。

缺口	影响	本该怎么做
没有任何基准测试	无法量化沙箱启动开销、调度吞吐、事件延迟。所有性能表述只能引用第三方文献	先测三个量：沙箱冷启动分布、单节点调度往返、事件端到端延迟
没有逃逸测试集	隔离设计的有效性只有论证，没有证据	按 M2 构造对抗提示词集，分层统计拦截率
没有故障注入	「可恢复」只在设计上成立，没被验证过	按 M1 做随机杀进程实验
类型判定无评估集	字段相似度是纯启发式，准确率未知	按 M4 标注端口对数据集
无对照系统	说不出「相比 X 好在哪」，只能说设计取舍不同	选 1 个同类系统，在同一组任务上比中断率与恢复成功率
无真实用户数据	循环轮数分布、中断率、PARTIAL 连接占比这些设计假设全部未经检验	埋点收集，用于校准 II.4 与 III.3 的取舍

为什么当时没做

这是一个以「功能完备的可部署系统」为目标的工程项目，不是以「回答某个问题」为目标的研究原型。目标不同，投入方向就不同：所有精力花在覆盖真实工作流所需的功能面与安全边界上。**这是一个明确的选择，不是疏忽**——但它也确实意味着这个系统目前只能作为「问题来源」，不能作为「结论来源」。

面试里的正确姿态

不要说「后续会补上评估」。说：「**这个系统的价值在于它暴露了四个我答不出数字的问题，其中 RQ1 和 RQ3 我认为可以做成独立课题。**」把工程缺口转化为研究入口，这才是委员会想听的。

V.3 · OPEN PROBLEMS

三个可以立项的问题

RQ 1

非确定性执行的重放：如何定义并度量「等价重放」？

问题：LLM 破坏了确定性重放的前提，但完全放弃重放意味着无法做回归测试。折中方案是对模型与工具调用做记录-回放，难点在于**请求指纹的定义**：指纹太严（含完整上下文）则任何提示词微调都导致全部失效；太松则会误命中错误的记录。

可研究形式：定义一族指纹函数（精确匹配 / 语义嵌入近邻 / 结构化槽位匹配），在真实执行轨迹上度量「命中率 × 误命中率」的帕累托前沿；并定义「执行轨迹等价」的形式化判据（如：可观测副作用序列的互模拟）。

为什么现在能做：轨迹数据可以从任意现有 Agent 平台采集，不依赖新系统。

RQ 2

动态图的静态可分析性：能否给出资源上界的保守估计？

问题：Agent 图的形状运行时才确定，因此无法在提交时回答「这次执行最多花多少钱 / 多长时间 / 调多少次工具」。而这恰恰是企业采用的前置条件——没有成本上界，没人敢把它接进生产流程。

可研究形式：把「有界展开 + 预算上限」的执行模型形式化，借鉴程序分析中的**资源上界推导**（如 amortized resource analysis），在给定循环预算、子 Agent 递归深度、工具调用上限的前提下，推导出保守的令牌与时间上界；再度量该上界相对真实消耗的松紧度。

难点：上界必须足够紧才有用。太松（比如“最多 100 轮 × 最大上下文”）在实践中等于没有。

RQ 3

类型协商的学习化：字段映射能否从历史连接中学出来？

问题：可视化编排里，端口 schema 的部分匹配需要给出候选字段映射。当前是字段名相似度启发式——没有 ground truth、没有评估集、没有准确率。

可研究形式：把它建模为**schema matching** 问题（数据集成领域的经典问题，已有评测传统），但引入两个新条件：① 训练信号来自用户在图上的历史确认/修正行为，是**隐式反馈**；② 需要给出 top-k 候选而非单一映射，评估指标应为 top-k 命中率与用户修正代价。

为什么有意思：它把一个纯工程的易用性问题，接到了数据集成领域已有的方法与评测体系上，且天然带有在线学习的反馈闭环。

V.4 · OPEN PROBLEMS

两个更偏系统安全的问题

RQ 4

人在回路的最优粒度：中断策略能否形式化为在线决策问题？

问题：审批门设得太密，用户放弃自动化；设得太疏，风险暴露。当前所有系统（包括本系统）都用**静态规则**决定哪些操作需要确认——按工具类型、按可逆性分档。但真实风险显然依赖上下文：同一个「写文件」，写在临时目录和写在配置目录完全不同。

可研究形式：建模为**带风险约束的在线决策**：每次工具调用面临「自动放行 / 请求确认」二选一，放行的期望损失取决于操作的不可逆性与上下文，确认的代价是固定的用户注意力开销。目标是在给定风险预算下最小化累计中断次数。可以用上下文赌博机（contextual bandit）框架，用用户的批准/拒绝作为反馈信号。

关键难点：损失是**非对称且长尾**的——绝大多数误放行没有后果，极少数造成不可逆损失。标准的后悔界分析在这种分布下意义不大，需要引入风险敏感的目标函数。

RQ 5

自修改 Agent 的可达状态空间：能否证明危险状态不可达？

问题：允许 Agent 修改自身配置（模型、工具集、权限策略）之后，出现一个新问题：**经过若干次自我修改，系统可能到达什么状态？**当前的防护是人工审批每次写操作——这只保证「每一步都被看过」，不保证「若干步之后不会到达危险配置」。人是无法通过审查单步变更来判断多步可达性的。

可研究形式：把 Agent 配置视为状态，把自进化操作视为状态迁移，把「能力单调收窄」等不变式视为安全属性，用**模型检验**回答可达性问题：给定初始配置与允许的操作集合，是否存在一条路径到达违反不变式的状态？如果存在，输出反例路径。

与 § II.6 的关系：这正是把那条不变式从「靠人工遵守」提升为「可自动验证」的路径。也是本系统里工程与形式化方法结合最紧的一个点。

如何选题

如果只能做一个：**RQ1** 门槛最低、数据最易得、结果对整个领域都有用（所有 Agent 平台都需要回归测试）；**RQ5** 理论深度最高，但需要先把配置空间和操作语义形式化，前期投入大。RQ3 介于两者之间，且能直接接上已有的 schema matching 评测传统。

最可能被问的十个问题

Q1 你这个系统解决的科学问题是什么？工程系统和研究有什么区别？

直说：**它现在是一个工程系统，不是研究工作**。它的学术价值在于暴露了四个我答不出数字的问题（§ V.2），其中两个我认为可以立项：非确定性执行的等价重放定义（RQ1）、自修改 Agent 的可达状态验证（RQ5）。**把工程系统当成问题来源而不是结论来源**，这是我对它的定位。

Q2 你说系统「可复现」，可 LLM 本身就不确定，这怎么成立？

这是我特意拆开的两条轴（§ II.3）。**我提供的是可追溯与可恢复（A 轴），不是重放（R 轴）**。而且要说清楚：连「结构重放」都不成立——只要条件分支消费了模型输出，两次执行走的节点集合都可能不同。把 A 轴成果说成「可复现」是最典型的过度宣称。

Q3 和 LangGraph 这类框架比，你的创新点在哪？

在计算模型上我没有创新，甚至更保守——它们允许带环状态图，我坚持顶层无环。**差异在于我同时承担了执行隔离与多租户治理**，而它们是库，把隔离留给调用方。这不是「更好」，是**处在设计空间的不同位置**（§ I.3）：我用表达力换可分析性与可运维性。

Q4 为什么不用现成的工作流引擎，比如 Temporal？

Temporal 的可靠性建立在**确定性重放**之上，要求 workflow 代码在重放时走出同一控制流。LLM 调用必须被包成 activity（其结果记入历史、重放时不重新执行）才能满足这个约束。这条路可行，但会把「图的形状」从可视化数据结构变成用户代码——而可视化编排恰恰是本系统的产品前提。**这是产品约束反过来限制了技术选型**，不是技术上做不到。

Q5 你说的「不变式」，有没有被形式化验证过？

没有。当前靠人工遵守与测试覆盖。**但它被写成了逐边可判定的形式**（ $\text{cap}(c) \subseteq \text{cap}(p)$ ），这是能否自动验证的前提。把它接到模型检验上就是 RQ5。我不会把「写成了不变式」说成「验证过了」。

Q6 系统这么复杂，怎么证明复杂度是必要的？

证明不了，只能给出反例：去掉不可变快照 → 无法归因；去掉能力收窄 → 权限正确性变成全局性质；去掉特权收敛 → 业务层漏洞升级为宿主逃逸。**我用的判据是「去掉它系统在哪里塌」，凡是去掉只是代码难看的，都不算抽象**（§ II 开篇）。这是论证不是证明，我知道区别。

Q7 你怎么知道你的设计假设是对的？

很多假设我**确实不知道对不对**，因为没有真实用户数据。比如：循环轮数分布（决定 II.4 那笔存储交换划不划算）、PARTIAL 连接的实际占比（决定类型协商值不值得做）、中断率随任务规模的增长阶。这三条都写在 § V.2，都是低成本可测的。

V.5 · Q&A · CONTINUED

三个更尖锐的

Q8 这个项目是 AI 写的，那你的贡献是什么？

承认，然后把话题拉到判断上：**代码生成解决的是「怎么写」，不解决「写不写、写哪一版、边界在哪」**。举三个具体决定——为什么 workflow 整体不自动重试而 Agent 任务重试四次（按副作用幂等性分层，不按重要性）；为什么监控趋势里队列深度返回空值而不是 0（没有历史时序就不能伪造）；为什么冷恢复清理失败要停住并留下脏资源（误放的代价是两个虚拟机抢同一块盘）。**这些是取舍，不是生成**。再补一句：我能准确说出自己系统的十几处缺陷（§ V.2 与实现册 F1），这比声称它完美更能说明我真的理解它。

Q9 如果给你三年时间重做，你会怎么做？

三件事，按优先级：

- ① **先建评估再建功能**。现在的处境是「系统能跑但说不出任何数字」，根源是评估从一开始就没进路线图。
- ② **抽出共享契约包**。端口类型这类跨端契约靠手工镜像在四处，已经实际漂移——这是本项目最明确的架构失误。
- ③ **把重放当成一等目标**。不是为了「可复现」这个好听的词，而是因为**没有重放就没有回归测试**，整个系统的演进只能靠人工验证，这在长期是不可持续的。

Q10 你对这个方向未来的判断是什么？

给一个可被反驳的判断：**Agent 系统的瓶颈正在从「模型能力」转移到「可问责性」**。模型能力在快速上升，但企业不敢把 Agent 接进关键流程，卡点是说不清「它做了什么、谁批准的、出事怎么归因、最多花多少钱」。

所以我认为下一阶段有价值的工作，是**把分布式系统与程序分析里已经成熟的东西——可恢复性、资源上界、状态可达性验证——重新适配到非确定性执行模型上**。RQ1、RQ2、RQ5 都在这条线上。

反驳方向：如果模型可靠到不需要人类问责（就像今天没人要求编译器给出可审计轨迹），这条线的价值就会大幅缩水。我认为短期内不会，但这是个真实的不确定性。

通用应答姿态

被问到不会的：**「我没做过，但我会这样设计实验：……」**比「不知道」和硬编答案都好。被问到自己系统的缺陷：**直接承认 + 说清影响范围 + 给修复路径 + 解释当时为什么没做**。最后一项最关键——「优先做了 X，因为 Y」比「没时间」有说服力得多。

V.6 · SCRIPT

三分钟怎么讲

「我做的是一个多智能体工作流编排平台。它要解决的核心矛盾是：**Agent 的价值来自自主性，而企业采用它的前提是可问责**——出了事要说得清做了什么、谁批准的、最多花多少钱。这两件事在计算模型上是冲突的：自主推理需要无界循环，可问责需要有限、可分析的结构。」

0:40 - 1:40 · 结构

「我的处理是不追求统一模型，而是**三层分离**：外层是确定性图，承载可追溯与可恢复；内层是无界的 Agent 回合，用预算和权限门截断；最底层是硬件级隔离，模型生成的代码在独立内核的微虚拟机里跑。

三层之间靠三条不变式缝合——**能力只能沿调用链单调收窄、不可信方永远不自证身份、运行时状态必须外置**。这三条的作用是把安全性从『每处都要记得遵守的规则清单』变成『逐边可判定的性质』。」

1:40 - 2:20 · 边界

「必须说清楚我**没有**提供什么：我提供的是可追溯与可恢复，**不是可重放**。只要条件分支消费了模型输出，连『两次执行走了哪些节点』都不保证一致。而且整个系统**没有做过任何基准测试和对照实验**——所有性能说法我只能引用文献，不能引用自己。」

2:20 - 3:00 · 问题在哪

「所以我把它定位成**问题来源**。做下来最让我在意的是两个问题：

第一，非确定性执行怎么做回归测试。没有重放就没法『改完重跑一遍比对』，而要重放就得定义『请求指纹』——太严则任何提示词微调都失效，太松则误命中。这个权衡曲线我认为是可以量化研究的。

第二，允许 Agent 改自己之后，若干步之后它能到达什么状态。人工审批每一步只保证每步被看过，不保证多步可达性安全——人是没法通过审查单步变更判断这个的。这本质上是状态可达性验证问题。

这两个问题都不依赖我这个系统才能研究，用公开的 Agent 平台采集轨迹就能开始。」

节奏提醒

三分钟里**不要出现任何表名、函数名、框架版本号**。技术细节等对方追问再给——主动给细节会让对方认为你只能讲细节。

工程系统的学术价值， 在于它暴露了 哪些还没被解决的问题。

所以这份文档最重要的不是前四部分，而是第五部分里那五个我答不出来的问题，和那份我没有做过的评估清单。

相关工作

均据官方文档、源码或论文核实；量级数字一律不引用未经自测的经验值

配套

实现细节参考册 53 页，仅在被追问具体机制时翻阅

重建

`node build.mjs design`